INDEX

S.No.	Date	Name of the Experiment	Mapping To Course Outcome	Grade	Signature
1)	8/7	Introduction to HTML and CSS	CO1	10	
2)	15/7	Static webpage using HTML and CSS	CO1	9	
3)	22/7	HTML Codes using audio & video tags	CO1	10	
4)	29/7	program using javascript	CO1	10	
5)	5/8	Implement modules in node JS	CO2	10	
6)	12/8	Develop mini Application Using Express & nodeJS	CO2	10	
7)	19/8	Code tantra[Static Webpage]	CO1	10	
8)	2/9	Code tantra(portfolio)	CO1	10	
9)	2/9	Code tantra [simple Calculator]	CO1	10	
10)	9/9	Create a functional & class	CO2	10	
11)	23/9	Implement http methods	CO2	10	
12)	30/9	Develop CRUD operation	CO3	10	

Anil Neerukonda Institute of Technology & Sciences
(Autonomous)
Affiliated to AU, Approved by AICTE & Accredited by NAAC with A+ Grade
Accredited by NBA (B.Tech - ECE, EEE, CSE, IT, MECH, Civil & Chemical)
Sangivalasa-531 162, Bheemunipatnam Mandal, Visakhapatnam District
Phone: 8712005999, 8712008222, Website: www.anits.edu.in email:

Department of Computer Science & Engineering (AI & ML)

Academic Year:	2026 - 2027	**Semester:**	3-1
Name of the Student:	G Manasha	**Roll Number:**	A24126552140
Name of the Lab:	FS Lab	**Timings:**	9:40 - 12:10

LAB WEEKLY ASSESSMENT SHEET

Date	Experiment /Week No	Observation Book				Total (15 Marks)	Record (10 Marks)	Faculty Remarks
		Theoretical Knowledge (4 Marks)	Programming Knowledge (4Marks)	Results (5Marks)	Viva-Voce (2Marks)			
1/7	01	4	4	5	2	15	10	
8/7	02	4	3	5	2	14	10	
15/7	03	4	4	5	2	15	10	
22/7	04	3	4	5	2	14	10	
29/7	05	4	4	4	2	14	10	
5/8	06	4	4	5	2	15	10	
12/8	07	4	4	5	2	15	10	
19/8	08	4	4	5	2	15	10	
2/9	09	4	4	4	2	14	10	
9/9	10	4	4	5	2	15	10	
23/9	11	4	4	5	2	15	10	
30/9	12	4	4	5	2	15	10	

Name of the Faculty: Dr. G. Prasanna Priya

Faculty Signature:

Introduction to HTML and CSS:

→ HTML [Hyper text Markup Language] is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents and interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→ CSS [Cascading style sheets] is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions and layouts. CSS also supports responsive design, allowing webpages to adapt to difference screen sizes and devices. CSS can be applied using inline, internal or external stylesheets.

→ HTML and CSS work together to develop Webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern Web development and enables the creation of organized, attractive and user-friendly Web interfaces.

Difference between MERN Stack & full Stack.

→ MERN Stack is a JavaScript-based technology stack used for developing modern Web applications. MERN stands for MongoDB is, Express.js, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework & API's, React.js is used to develop frontend user interface, and node.js provides the server side runtime environment.

→ Full Stack Development refers to development of both the frontend & backend of a Web application along with database integration.

HTML Components:

1. HTML tags - Define the structure and elements of a webpage.
2. Headings - used to display titles & headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag.
4. Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag.
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. tables - used to organize data in rows and column.
8. forms - used to collect user input using elements such as text boxes, buttons and checkboxes.
9. Div & Span - used to group and organize webpage content
10. Semantic Elements - Elements such as <header>, <nav>, <selection>, <article>, <footer> provide meaningful webpage structure.

CSS Components:

1. Selectors - Select HTML elements to Apply Styles
2. properties - Define the visual characteristic of elements.
3. values - Specify the setting of CSS property
4. Colors - Define text, background, border Colors.
5. Fonts - Color Control font family, size, style and weight.
6. Backgrounds - Set background Colors & images
7. Borders - Define the border style, width and radius.
8. Margin - Controls the space outside an element.
9. Padding - Controls the space inside element
10. Layout - Controls the arrangement of elements using flexbox, grid and positioning

HTML Vs HTML5

HTML	HTML5
• Older version of HTML	• Latest major version of HTML.
• Limited Multimedia Support.	• Supports audio and video.
• Uses external plugins for multimedia	• no external plugs needed
• Has fewer semantic tags	• provides semantic tags
• no <canvas> element	• Supports <canvas>
• Limited web storage	• Supports local storage

CSS Vs CSS3

CSS	CSS3
• Earlier version of CSS	• Advanced version of CSS
• Basic Styling features	• provides advanced Styling features.
• Limited animation	• Supports animations and transitions.
• not flexible layout System.	• Supports flexbox and grid.
• Limited visual effects	• Supports shadows, gradients, effects.
• no media queries	• Supports media queries.

Experiment 02

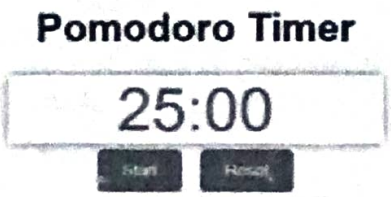
Design a static web page using HTML & CSS

Pomodoro timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title Pomodoro timer</title>
    <style>
        body{
            font-family : Arial;
            background : #F5F5F5;
            text-align : center;
            padding : 50px;
        }
        .timer{
            width: 300px;
            margin: auto;
            padding: 30px;
            background : White;
            border-radius: 15px;
            box-shadow : 0 0 10px gray;
        }
        h1{
            color : #e74c3c;
        }
        .time{
            font-size: 50px;
            margin : 20px;
        }
```

```
        button {
            padding: 10 px 20px;
            margin : 5 px;
            bolder : none;
            bolder-radius: 5 px;
            background : #e74c3c;
            color : white;
        }
    </style>
</head>
<body>
    <div class = "Timer">
        <h1> Pomodoro timer </h1>
        <div class = "time"> 25:00 </div>
        <button> Start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

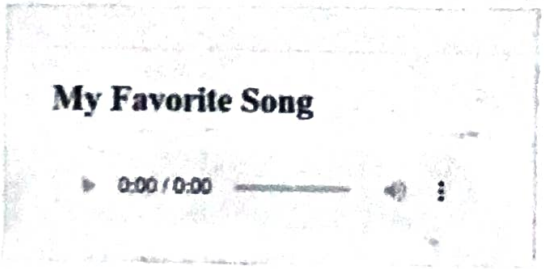
O/P:



ii) Design an html code using audio tags

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body>
    <h2> My favorite song </h2>
    <audio controls>
        <source src="song.mp3" type="audio/mpeg">
        your browser does not support the audio tag
    </audio>
</body>
</html>
```

Op: Displays an Audio player with play, pause and volume controls.



i) Design a html code using video tags.

```
<!DOCTYPE html>
<html>
<head>
    <title> video Example </title>
</head>
<body>
    <h2> My video </h2>
    <video width = "400" controls>
        <source src = "video.mp4" type = "video/mp4"
    </video>
</body>
</html>
```

O/P: Displays a video player with play, pause, volume and fullscreen controls.

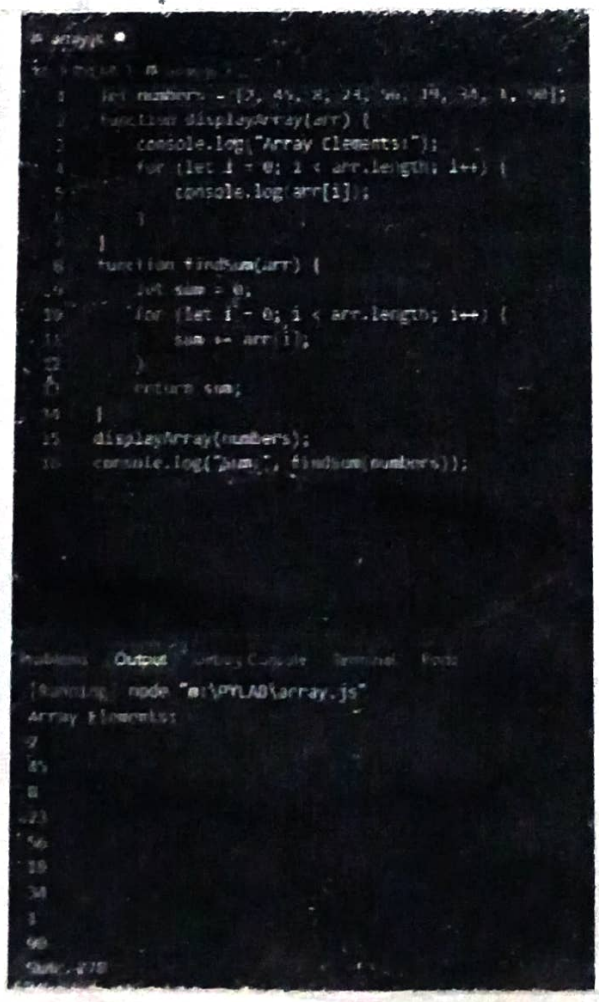


Experiment-3

3. Write a program Using Javascript [arrays, functions].

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray(arr) {
    console.log("Array Elements:");
    for (let i=0; i<arr.length; i++) {
        console.log(arr[i]);
    }
}
function findSum(arr) {
    let sum = 0;
    for (let i=0; i<arr.length; i++) {
        sum += arr[i];
    }
    return sum;
}
```

O/P:



Experiment-04

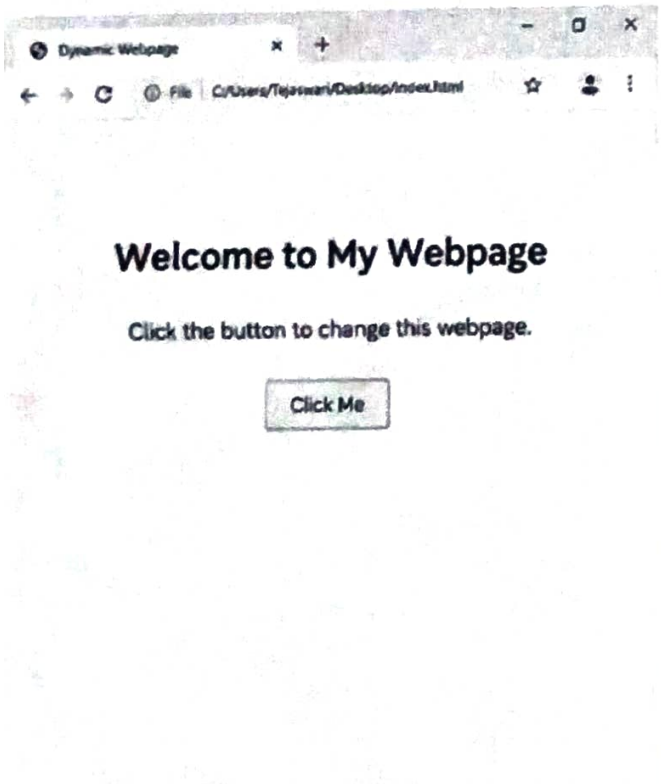
4. Develop dynamic webpage using java script

• program:

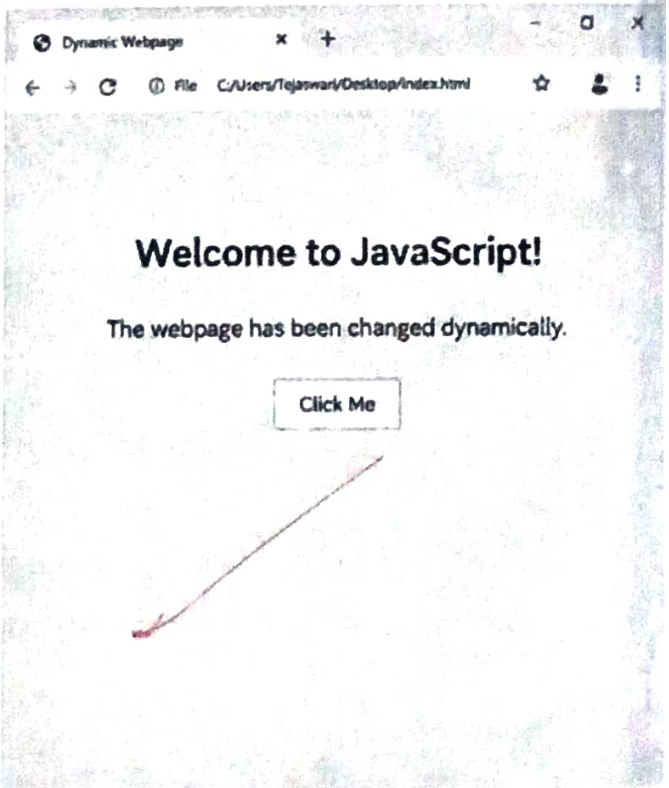
```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Webpage </title>
    <style>
        body{
            text-align : center;
            font-family : Arial;
            margin-top : 100px;
        }
        button{
            padding : 10px 20px;
            font-size : 16 px;
            cursor : pointer;
        }
    </style>
    </head>
    <body>
    <h1 id = "heading"> Welcome to My Webpage</h1>
    <p id = "message"> click the button to change
                        this Webpage</p>
    <button onclick = "changePage()"> click Me</button>
    <script>
        function changePage(){
```

```
document.getElementById("heading").
    innerHTML = "Welcome to javascript!";
document.getElementById("message").
    innerHTML = "The Webpage has been
changed dynamically";
document.body.style.backgroundColor =
                        "lightblue";
    }
</script>
</body>
</html>
```

O/P:



Before clicking the button



After clicking the button

Experiment-5

1) Implement Modules in NodeJS.

- In Node.js you can implement modules using two distinct systems, CommonJS (CJS), which is traditional default, and ESModules (ESM), the modern JavaScript standard.

1. The Common Method (default):

CommonJS used Module.exports to share code and require() to import it. This system loads modules synchronously.

a) Create the module (math.js):

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
        add,
        subtract
      };
```

b) Use the module (app.js):

```
const math = require('./math');
console.log(math.add(10,5));
console.log(math.subtract(10,5));
```

Output: 15

5

2) The ES Modules Method (modern standard):

ES modules use export to share code & import to use it. to make node.js recognize ES modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a) Create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b
const calculatorName = "SuperCalc";
export default calculatorName;
```

b) Use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';
    console.log(calculatorName)
    console.log(multiply(4,2));
    console.log(divide(4,2));
```

o/p: SuperCalc

8

2

3) Using Built-in core modules:

Node.js also comes equipped with dozen of built-in core modules (like fs for file system & path for file routing) that don't need information.

We will load them by module names:

```
// Common JS style
const fs = require('fs');
// ES modules style
import fs from 'fs';
```

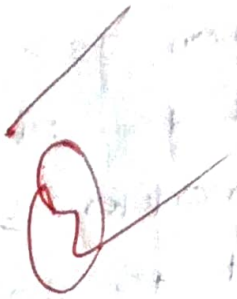
Here are some of the most commonly used ones:

1. fs = file system and client
2. http = HTTP servers and client
3. path = file path utilies
4. OS = Operating System Utilities
5. Events = event handling
6. Util = Utility functions
7. Stream = Stream handling
8. Crypto = Cryptographic function
9. URL = URL parsing
10. query string = URL query string handling.

Eg: Simple HTTP Server

```
http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type':
                        'text/html'});
    res.end('Hello World!');
}).listen(808);
```

O/P: Hello World!



Experiment-6

6) Develop mini Application using Express & nodeJs.

Simple mini application using node JS + Express JS

Student Management System:

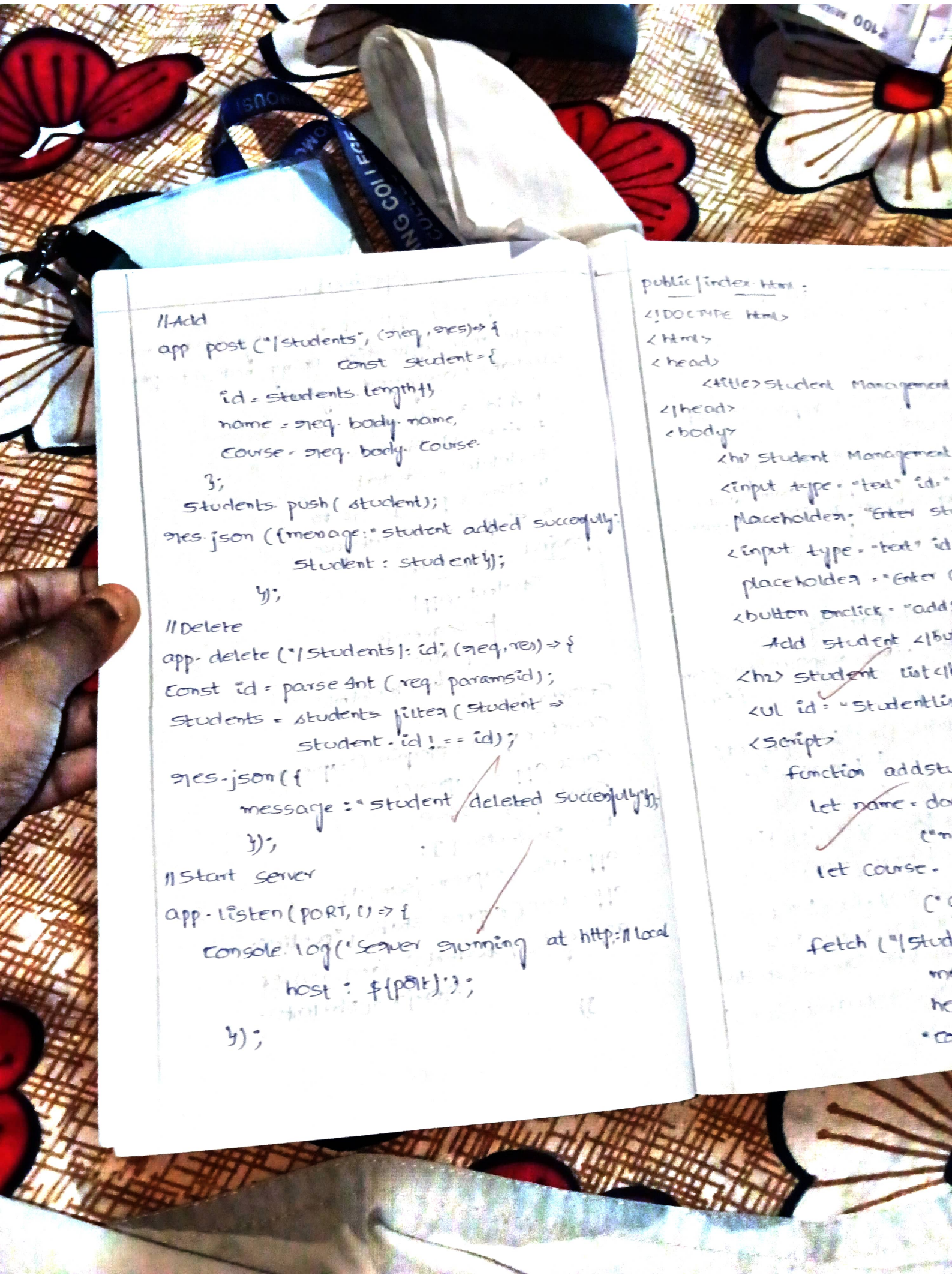
1. Add a Student
2. View all Students
3. delete a Student
4. Uses Express js routes
5. Simple HTML interface.

Project Structure:

```
Student-app/
    ↳ server.js
    ↳ public
        ↳ index.html
```

Server.js:

```
Const express = require("Express");
Const app = Express()
Const port = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
app.get("/students", (req, res) => {
                res.json(students);
});
```



```
//Add
app.post("/students", (req, res) => {
        const student = {
    id = students.length+1;
    name = req.body.name,
    course = req.body.course
  };
  students.push(student);
res.json({message:"student added succesfully",
        student : student});
    });
//Delete
app.delete("/students/:id", (req,res) => {
const id = parseInt(req.params.id);
students = students.filter(student =>
        student.id !== id);
res.json({
    message :"student deleted succesfully"});
    });
//Start server
app.listen(PORT, () => {
  console.log('Server running at http://local
      host : ${PORT}');
  });
```

public/index.html :

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management
</head>
<body>
    <h1> Student Management
    <input type="text" id="
     placeholder="Enter st
    <input type="text" id
     placeholder ="Enter
    <button onclick="add
      Add student </bu
    <h2> Student list</h
    <ul id="studentli
    <script>
      function addst
        let name = do
                ("n
        let course =
                ("
        fetch ("/stud
                m
                he
                "co
```

```
//Add
app.post("/students", (req, res) => {
        const student = {
    id = students.length+1;
    name = req.body.name,
    course = req.body.course
  };
  students.push(student);
res.json({message:"student added succesfully",
        student : student});
    });
//Delete
app.delete("/students/:id",(req,res) => {
const id = parseInt(req.params.id);
students = students.filter(student =>
        student.id !== id);
res.json({
    message :"student deleted succesfully"});
    });
//Start server
app.listen(PORT, () => {
  console.log('server running at http://localhost:${PORT}');
  });
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
</head>
<body>
    <h1> Student Management System</h1>
    <input type="text" id="name"
    placeholder="Enter student name">
    <input type="text" id="Course"
    placeholder="Enter Course">
    <button onclick="addStudent()">
      Add Student</button>
    <h2> Student List</h2>
    <ul id="Studentlist"></ul>
    <script>
      function addstudent(){
        let name = document.getElementById
                  ("name").value;
        let course = document.getElementById
                  ("course").value;
      fetch("/students",{
                  method:"POST",
                  headers:{
                "Content-Type":"application/json"},
```

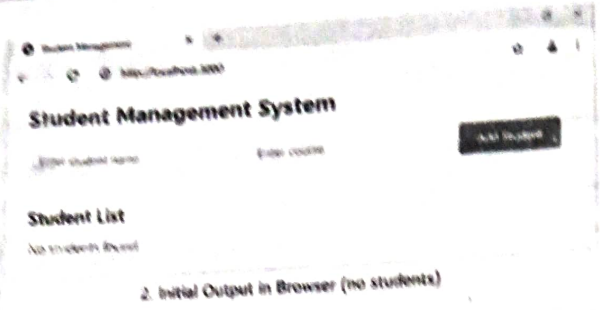
```
    body: JSON.stringify({
        name: name,
        course: course
    })
  })
  .then(response => response.json())
  .then(data => {
        alert(data.message);
document.getElementById("name").value = "";
document.getElementById("course").value = "";
    displayStudents();
  });
}
function displayStudents() {
fetch("/students").then(response => response.
                                    json())
.then(students => {
  let list = document.getElementById(
                        "studentlist");
  list.innerHTML = "";
students.forEach(student => { let li =
document.createElement("li");
    li.innerHTML = student.id + " - " +
                   student.name + " - " +
                   student.course + "<button
onclick = "deleteStudent(" + student.id + ")">
        Delete </button>";
```

```
list.appendChild(li);
            });
        });
    }
function deleteStudent(id){
    fetch("/students/"+id,{method:"DELETE"})
.then(response => response.json())
.then(data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents();
</script>
</body>
</html>
```

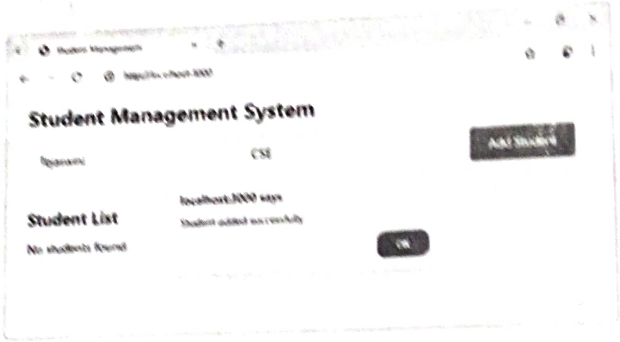
O/P:



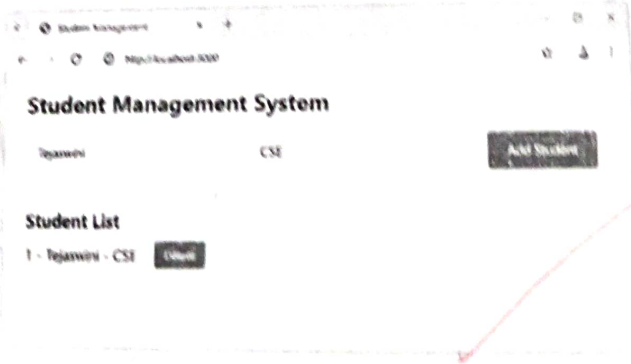
1. Terminal Output (after running `node server.js`)



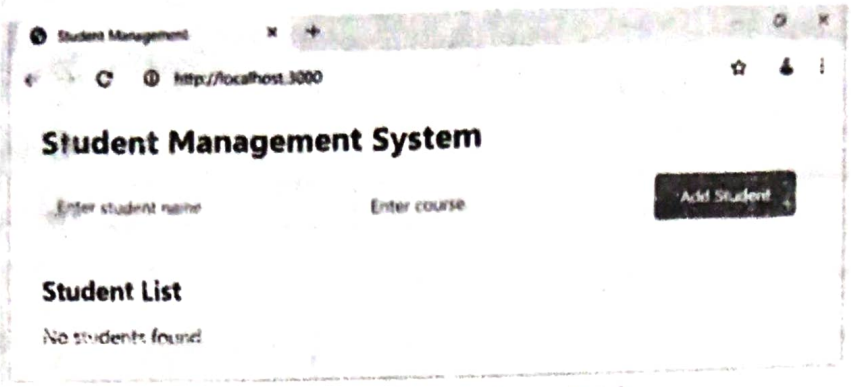
2. Initial Output in Browser (no students)



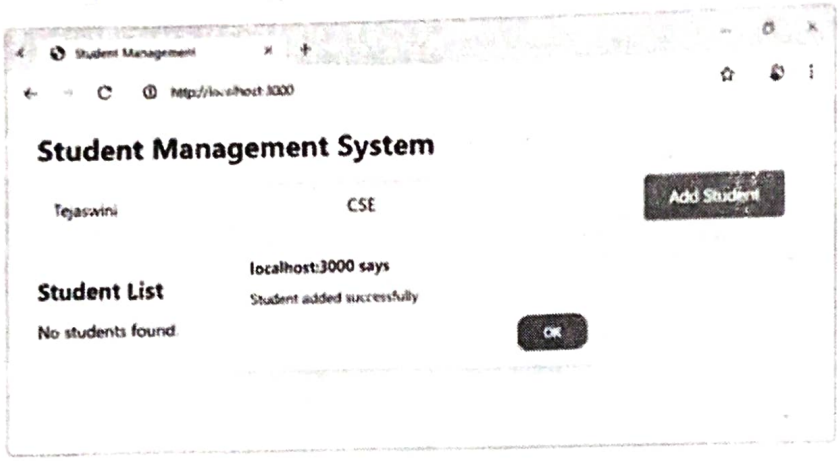
3. After adding a student (alert message)



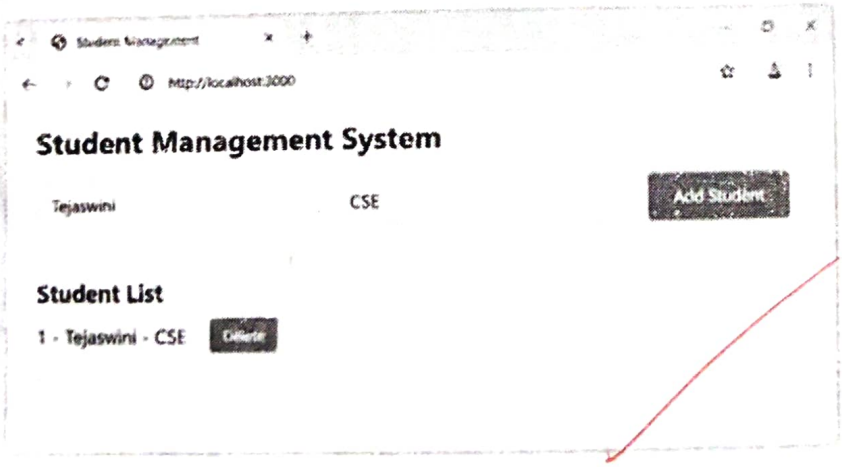
4. Student list after adding one student



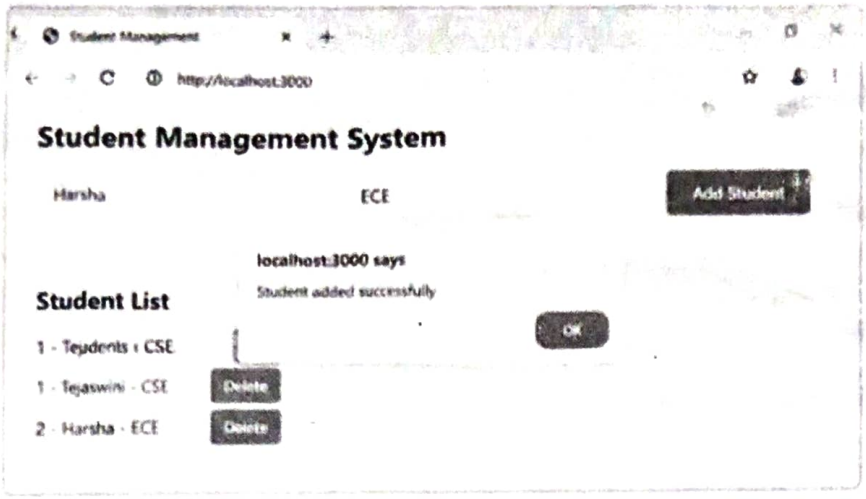
2. Initial Output in Browser (no students)



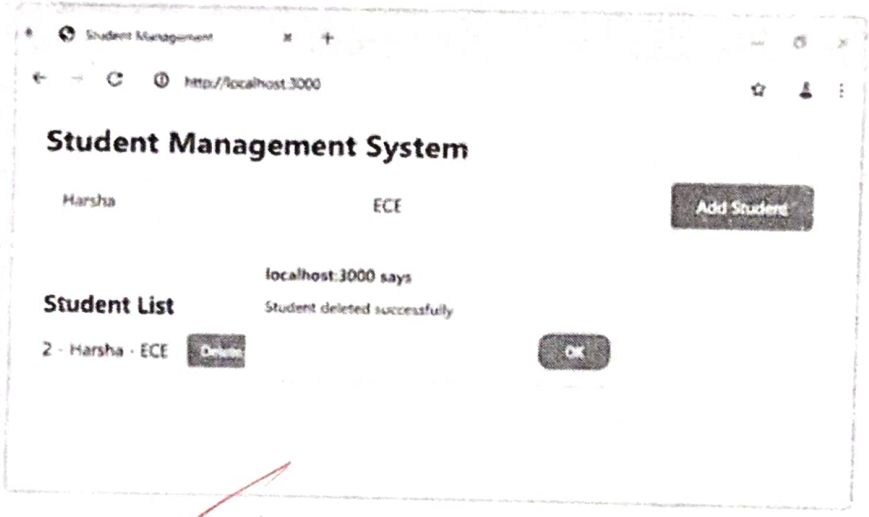
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 07

Q) Create a functional Component and class component using react.

- **Aim:** to develop a react application using functional components and class components to display student details, manage state, handle button clicks and dynamically update data on a web page.
- **functional component:**

A functional component is a javascript function that returns JSX to display the user interface. It uses React hooks such as usestate to manage state and update the webpage dynamically.

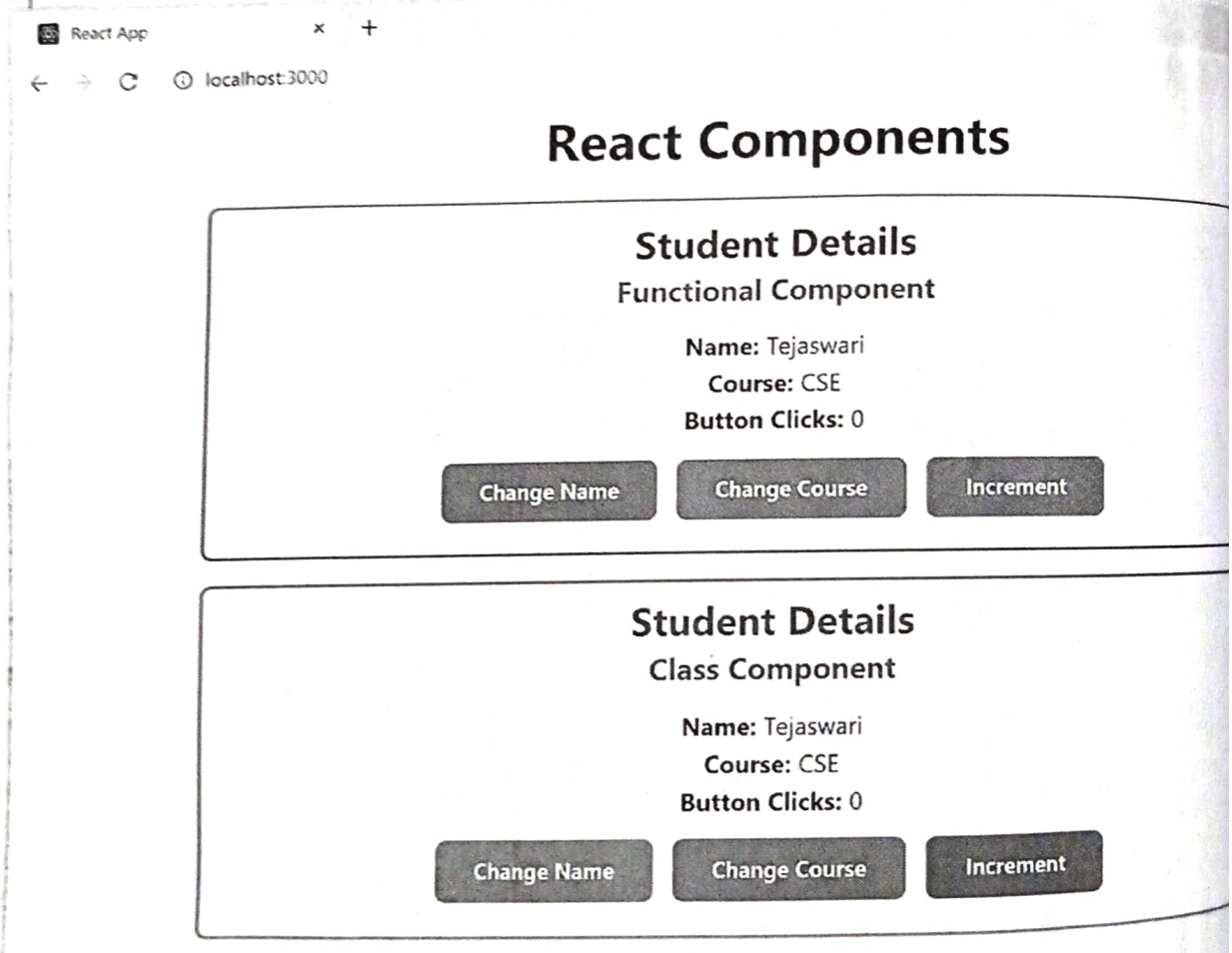
program:

```
import React, {useState} from
'react';
function StudentFunctional(){
        const [name, setName] = useState("Manasha");
        const [course, setCourse] = useState("CSM");
        const [count, setCount] = usestate(0);
        const changeName = () => {
            setName("Manshu");
        };
```

```
const changeCourse = () => {
        setCourse ("CSD");
    };
const incrementCount = () => {
        setCount(count+1);
    };
return (
  <div style = {{
      border: "2px solid blue",
      padding: "20px",
      margin: "20px",
      borderRadius: "10px",
      textAlign : "center"
    }}>
    <h1> Student Details </h1>
    <h2> Functional Component </h2>
    <p> <b> Name: </b> {name} </p>
    <p> <b> Course: </b> {Course} </p>
    <p> <b> Button clicks: </b> {Count} </p>
    <button onclick = {changeName}>
          changeName </button>
    <button onclick = {changeCourse}> change
          Course </button>
```

```
<button onClick={incrementCount}>Increm
        </button>
    </div>
);
```

Output: When the user clicks the button, the name, course & count change dynamically without reloading the webpage.



- Class Component:

A class Component is an ES6 JavaScript class that Extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

program:

```
import React, {Component} from
'react';
class studentclass extends Component {
    constructor(props) {
        super(props);
        this.state = {
            name: "Manasha",
            Course: "Computer Science",
            count: 0
        };
    }
    changeName = () => {
        this.setState({
            name: "Mamsha"
        });
    };
    changeCourse = () => {
        this.setState({
            Course: "CSM"
        });
    };
```

```
incrementCount = () => {
    this.setState((prevState) => ({
        count: prevState.count + 1
    }));
};
render(){
    return(
        <div style={{
            border: "2px solid green",
            padding: "20px",
            margin: "20px",
            borderRadius: "10px",
            textAlign: "center"
        }}>
        <h1> Student Details </h1>
        <h2> class Component </h2>
    <p> <b> Name: </b> {this.state.name} </p>
    <p> <b> Course: </b> {this.state.course} </p>
    <p> <b> Button clicks: </b> {this.state.count} </p>
    <button onClick={this.changeName}>
                changeName </button>
    <button onClick={this.changeCourse}>
                changeCourse </button>
    <button onClick={this.incrementCount}>
                Increment </button>
        </div>
```

Experiment-08

8) Implement http methods using JSON server

Aim: to implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON Server to perform CRUD (Create, Read, update & Delete) operations on student records.

Theory: JSON Server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from Scarch.

HTTP methods are used to communicate b/w a client and a Server.

- GET - Retrieves data from the server
- POST - Adds new data to the Server
- PUT - Replaces an existing record
- PATCH - Updates Selected fields of a record
- DELETE - Removes a record from the server.

a) Create a JSON Server:

Step 1: Initialize the project

Open the VS Code terminal & execute the following commands:

```
mkdir JSON server
cd JSON Server
npm init -y
npm install json Server
```

step: create db.json

Create a file named db.json and add the following student data

```
{
  "students": [
    {
      "id": "1",
      "name": "Manisha",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "2",
      "name": "Mokshu",
      "course": "CSM",
      "marks": 90
    },
    {
      "id": "3",
      "name": "Teju",
      "course": "IT",
      "marks": 88
    }
```

Step 3: Start the JSON Server

For the current JSON Server version, execute

"npx json-server db.json"

The Server Should ...
http://localhost:...
The student ...
http://localhost:...

2. Implementation ...

of GET Method: The ... retrieve student ...

Code:

```
fetch("http://l...
  .then(respons...
  .then(data...
      consol...
    })
  .catch(er...
      ca...
    })
```

Step2: Create db.json

Create a file named db.json and add the following student data.

```
{
  "students": [
    {
      "id": "1",
      "name": "Manasha",
      "Course": "CSM",
      "marks": 90;
    },
    {
      "id": "2",
      "name": "Mokshu",
      "Course": "CSM",
      "marks": 90
    },
    {
      "id": "3"
      "name": "Teju",
      "Course": "IT",
      "marks": 88
    }
```

Step 3: Start the JSON Server

for the current JSON Server version, Execute

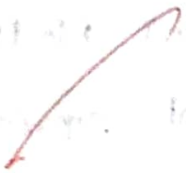
npx json-server db.json

The Server Start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

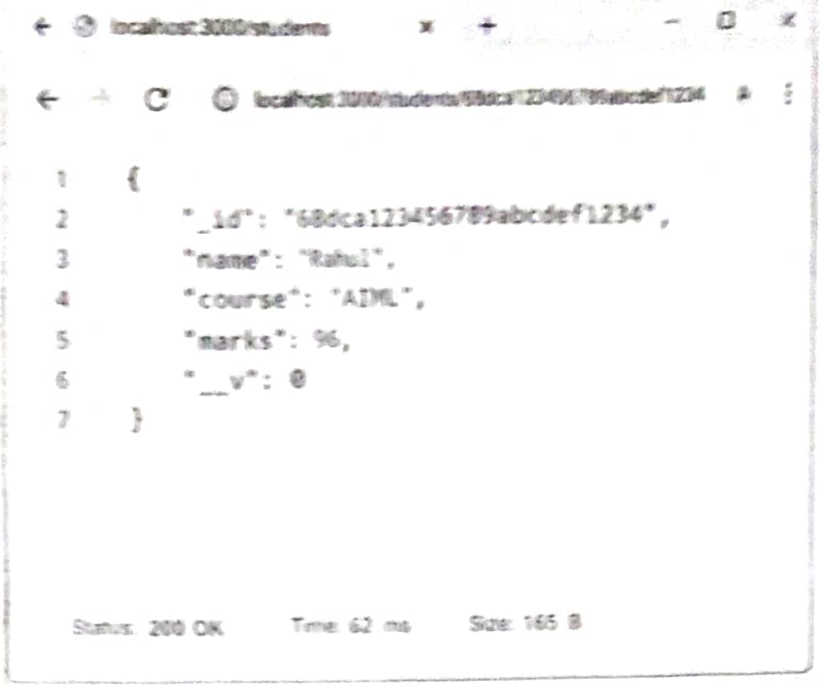


2. Implementation of HTTP methods

a) GET Method: The GET method is used to retrieve student records from JSON server.

Code:

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  })
```



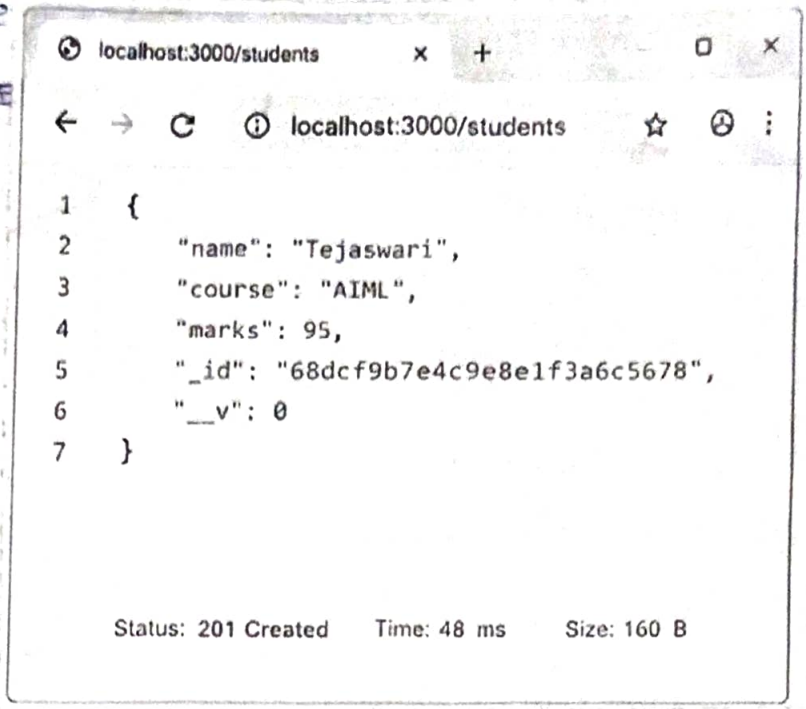
B. POST Method: The POST Method is used to insert a new student record into the server.

Code:
```
fetch ('http://localhost:3000/students', {
    method : 'POST',
    headers : {
      'Content-Type' : 'application/json'
    },
    body : JSON.stringify({
        name : 'Rahul',
        course : 'CSE',
        marks : 92
    })
})
```

- then (response => response.json())
- then (data => {
    console.log(data);
})

• Catch (e
cons
});

O/P:

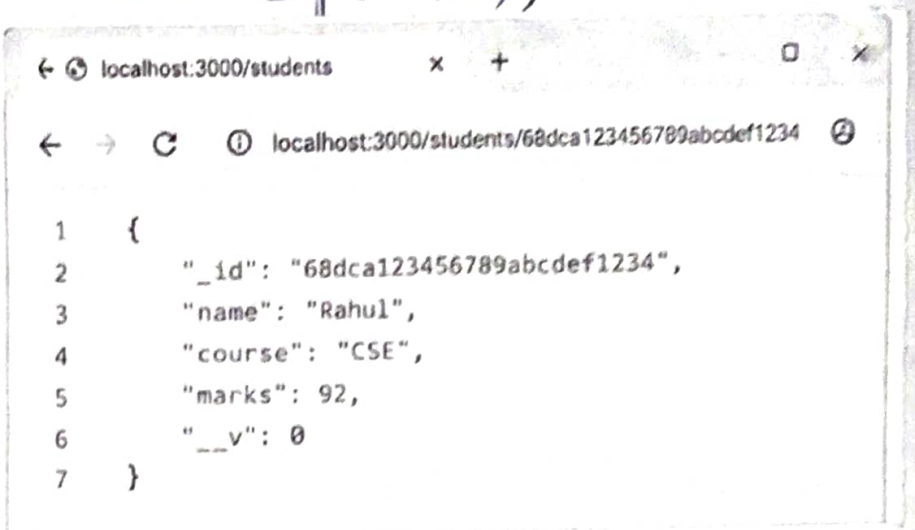


c) PUT Method: The PUT method is used to replace an existing student record with updated data.

Code:
```
fetch('http://localhost:3000/students/1',{
    method : 'PUT',
    headers : {
        'Content-Type' : 'application/json'
    },
    body : JSON.stringify({
        name : 'Tejaswari',
        course : 'AIML',
        marks : 95
    })
})
.then (response => response.json())
```

```
.then (data => {
    console.log(data);
})
.catch (err => {
    console.log(err);
});
```

O/p:

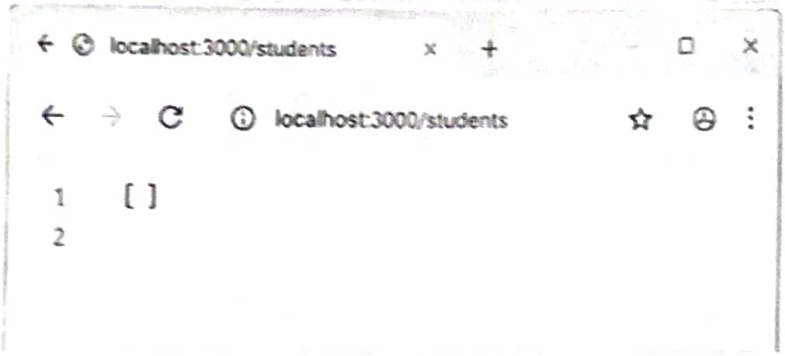


D) PATCH Method: The PATCH method is used to update only selected fields of an existing student record.

Code:
```
fetch ('http://localhost:3000/students/2', {
    method : 'PATCH',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify ({
        marks: 96
    })
})
.then (respone => respone.json())
.then ( data => {
```

```
console.log(data);
});
.catch(error => {
    console.log(error);
});
```

O/P:

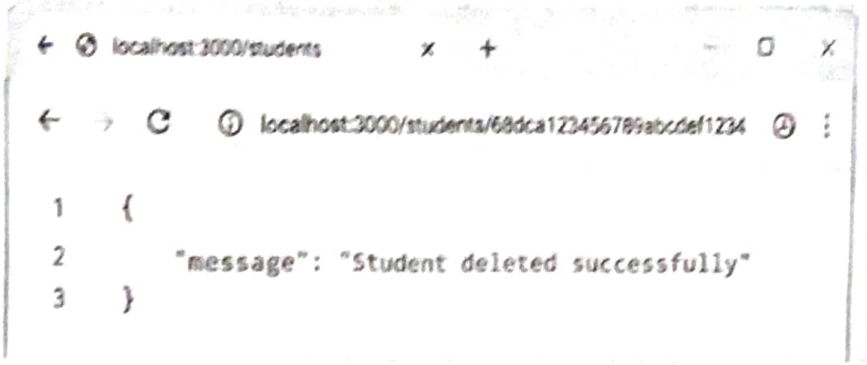


E) DELETE Method: The DELETE method is used to remove an existing student record from the server.

Code:

```
fetch('http://localhost:3000/students/3',{
    method:'DELETE'
})
.then(response=>{
    if(response.ok){
        console.log("Student deleted succes");
    }
})
.catch(error=>{
    Comdo.log(error)
});
```

O/P:



- After executing the POST, PUT, PATCH & DELETE operations in sequence, the remaining student

O/P:



Thus, HTTP methods GET, POST, PUT, PATCH and DELETE were succesfully implemented using JSON server. The CRUD operation were performed on student records. The data was retrieved, inserted, updated & deleted using HTTP requests.

Experiment-09

9) Develop CRUD operations Using MongoDB.

Aim: to develop a web application Using Node.js, express.js and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NOSQL database that stores data in flexible, JSON like documents. CRUD operations are used to manage data in a database.

Mongoose is an object Data Modeling [ODM] library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1) Installation and project setup:

```
mkdir MongoDB-CRUD
cd MongoDB-CRUB
npm init -y
npm install express mongoose
```

2) program: server.js

```
const express = require ('Express');
const mongoose = require ('mongoose');
Const app = express();
```

```
Const PORT = 3000;
app.use(expren.json());
mongoose.connect('mongodb://127.0.0.1:
                  27017/college')
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
//Create Student
Const studentSchema = new
mongoose.schema({
        name: {
          type: String,
          required: true
        },
        Course: {
          type: String,
          required: true
        },
        marks: {
          type: Number
          required: true
        }
});
```

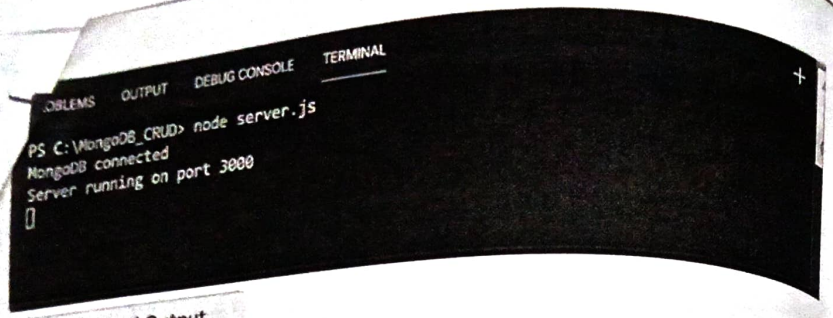
```
// create Student model
  const Student =
     mongoose.model ('Student', studentSchema)
// create : Add
 app.post ('/students', async (req, res) => {
     try {
       const student = new student (req.body);
       const savedStudent = await student.
                                    save();
 res.status (201).json (saved student);
 } catch (error) {
     res.status (400).json ({message:
                  error : message});
               }
           });
//READ : Get all students
app.get ('/students', async (req, res) => {
     try {
       const students : await
       Student.find ();
         res.status(500).json ({message: error.
                                  message});
     }
  });
```

```
// READ: Get Student by ID
app.get('/students/:id', async (req, res) => {
    try {
        const student: await
        Student.findById (req.params.id);
        if (!student) {
            return res.status(404).json ({
              message: "student not found"
            });
        }
      res.status (200).json(student)
    } catch (error) {
        res.status (400).json({ message:
        error.message });
    }
});

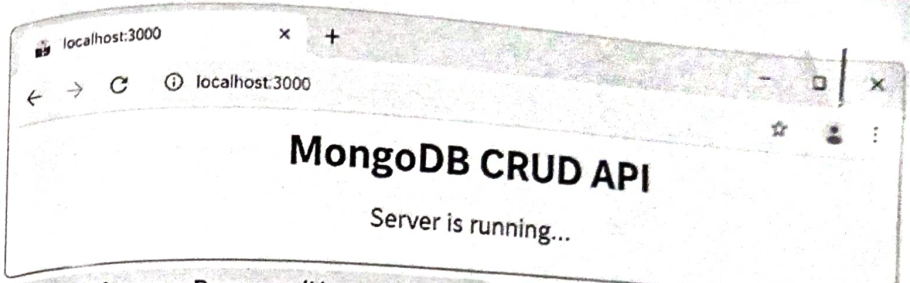
// UPDATE
app.put('/students/:id', asyn(req,res)=>{
    try {
Const student: await
Student.findByIdAndUpdate (
        req.params.id,
        req.body,
```

```
    {new: true, runValidators: true}
  );
  if (!student) {
    return res.status(404).json({
      message: "Student not found"
    });
  }
  res.status(200).json(student);
} catch (error) {
  res.status(400).json({ message:
    error.message});
  }
});

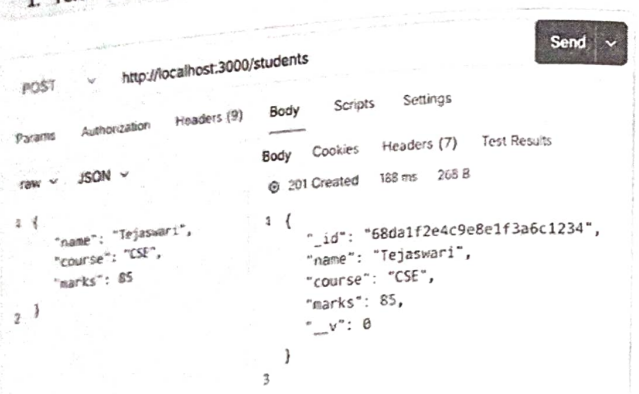
//DELETE
app.delete('/students/:id', async(req,res)=>{
  try{
    const student = await student.
      findByIdAndDelete(
        req.params.id
      );
    if (!student) {
      return res.status(404).json({
        message: "student not found"
      });
    }
```



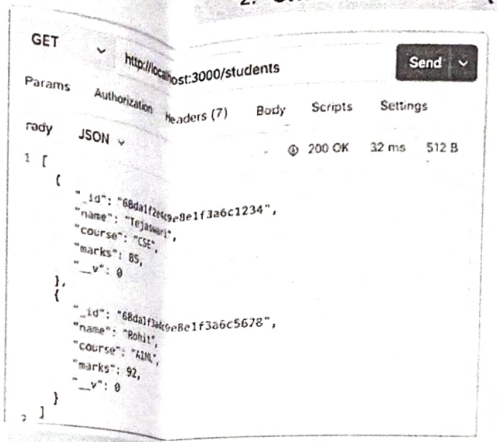
1. Terminal Output



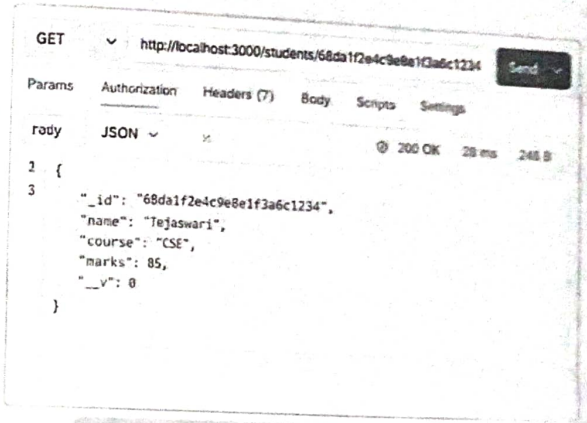
2. Chrome Browser (Home Page)



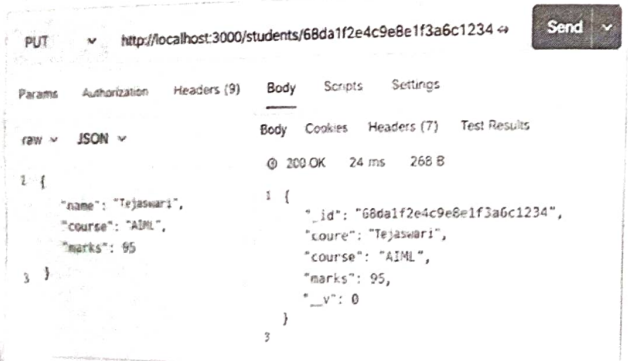
3. Create (POST) – Add Student



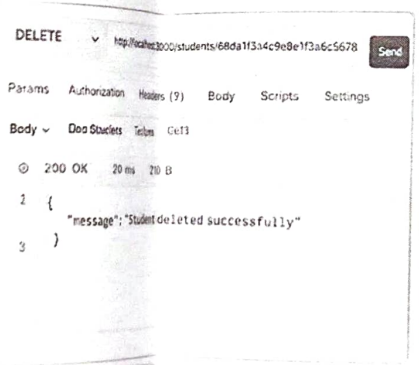
4. Read (GET) – Get All Students



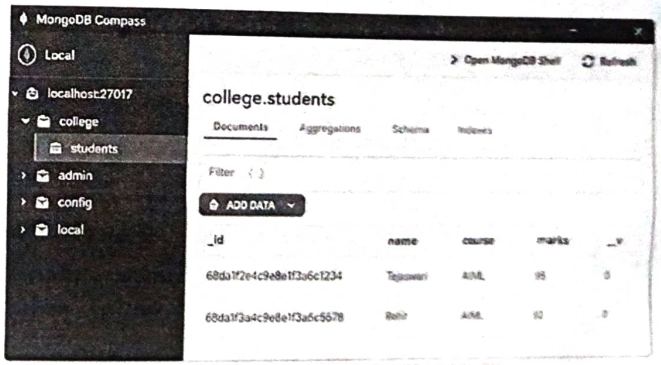
5. Read (GET) – Get Student by ID



6. Update (PUT) – Update Student



7. Delete (DELETE) – Remove Student



8. MongoDB Compass – Database View